

BUG-001 · Docker cannot pull Langflow image — lookup auth.docker.io: no such host

Field	Detail
Issue	<code>docker run langflowai/langflow:latest</code> failed to download the image with a DNS lookup error.
Found Date	2026-07-23
Status	Closed
Severity	HIGH
File	Environment (Docker Desktop / WSL2), not project code
Description	First run of <code>docker run -p 7860:7860 langflowai/langflow:latest</code> reported Unable to find image locally (normal), then failed with lookup <code>auth.docker.io: no such host</code> . Docker could not resolve Docker Hub's auth/registry hosts.
Root Cause	Not Langflow and not a missing image. Windows host DNS actually resolved fine (<code>nslookup auth.docker.io</code> and <code>registry-1.docker.io</code> both returned addresses, and <code>curl https://auth.docker.io/token</code> succeeded), so the failure was stale DNS state inside Docker Desktop's internal WSL2/Linux VM rather than on the host.
Solution	Quit Docker Desktop completely, then <code>ipconfig /flushdns</code> and <code>wsl --shutdown</code> (Admin), restart Docker Desktop, and retry <code>docker pull hello-world</code> then <code>docker pull langflowai/langflow:latest</code> . If it recurs, pin explicit DNS (<code>{"dns":["1.1.1.1","8.8.8.8"]}</code>) in Docker Engine settings. The pull subsequently succeeded. Ultimately superseded by moving off Docker entirely — see ADR-001.
Date Resolved	2026-07-23

BUG-002 · Langflow container exits on startup — Username and password must be set

Field	Detail
Issue	After the image pulled successfully, the Langflow container shut down immediately instead of serving on port 7860.
Found Date	2026-07-23
Status	Closed
Severity	MEDIUM
File	Environment (Docker container config), not project code
Description	Container logs ended with <code>Missing credentials: username=langflow, password=not set</code> followed by <code>ValueError: Username and password must be set</code> . The PyTorch and CORS lines in the same log were only warnings and did not cause the shutdown.
Root Cause	The current <code>langflowai/langflow:latest</code> image ships with login enabled by default and refuses to start without superuser credentials. The failed container also still owned the name <code>langflow</code> , blocking a clean re-run.
Solution	<code>docker rm langflow</code> , then start with auth env vars: <code>-e LANGFLOW_AUTO_LOGIN=false -e LANGFLOW_SUPERUSER=langflow -e LANGFLOW_SUPERUSER_PASSWORD="<strong-password>"</code> (the legacy default password <code>langflow</code> is rejected). For a private local experiment, <code>-e LANGFLOW_AUTO_LOGIN=true</code> bypasses the login screen. See Research topic 4. Superseded in practice by the local install (ADR-001).
Date Resolved	2026-07-23

BUG-003 · python3.14 uv venv fails — can't open file '...\uv'

Field	Detail
Issue	Attempt to create a virtual environment errored with a file-not-found on <code>uv</code> .
Found Date	2026-07-23
Status	Closed
Severity	LOW
File	Environment (shell command), not project code
Description	Running <code>python3.14 uv venv</code> produced <code>can't open file 'C:\Users\LOQ\Desktop\Projects\Langflow-learning-project\uv': [Errno 2] No such file or directory</code> .
Root Cause	<code>python3.14 uv venv</code> tells Python to <i>execute a script file named uv</i> , so Python searched for <code>.\uv</code> and failed. <code>uv</code> is a standalone tool, not a Python script argument.
Solution	Invoke <code>uv</code> directly: <code>uv venv .venv --python 3.12</code> then <code>.\venv\Scripts\Activate.ps1</code> . See ADR-003 for the Python version choice.
Date Resolved	2026-07-23

BUG-004 · uv install langflow — unrecognized subcommand 'install'

Field	Detail
Issue	Package install command rejected by <code>uv</code> .
Found Date	2026-07-23
Status	Closed
Severity	LOW
File	Environment (shell command), not project code

Description	<code>uv install langflow</code> returned <code>error: unrecognized subcommand 'install'</code> with the hint <code>a similar subcommand exists: 'uv pip install'</code> .
Root Cause	<code>uv</code> has no top-level <code>install</code> ; package installation goes through the pip-compatible interface.
Solution	Use <code>uv pip install langflow</code> (later pinned — see BUG-005 and ADR-002).
Date Resolved	2026-07-23

BUG-005 · `litellm==1.93.0` fails to build on Windows — `link.exe` not found

Field	Detail
Issue	<code>uv pip install langflow</code> (which resolves <code>langflow 1.11.0</code>) failed while building <code>litellm==1.93.0</code> from source.
Found Date	2026-07-23
Status	Closed
Severity	HIGH
File	Environment (dependency build), not project code
Description	Resolution pulled <code>litellm==1.93.0</code> (via <code>langflow 1.11.0</code> → <code>langflow-base[complete] 0.11.0</code>). The wheel build ran <code>maturin</code> , auto-installed a temporary Rust toolchain, began compiling the <code>pyo3</code> native crates, then failed: <code>error: linker link.exe not found / the msvc targets depend on the msvc linker but link.exe was not found</code> .
Root Cause	LiteLLM 1.93.0 ships Linux wheels but no <code>win_amd64</code> wheel , so Windows falls back to compiling the Rust/ <code>pyo3</code> <code>python-bridge</code> from source, which requires the MSVC C++ linker (<code>link.exe</code>) from Visual Studio Build Tools — not installed. This was the real blocker; the earlier suspicion that Python 3.14 was at fault was a red herring (the failure reproduced on 3.12 too).
Solution	Pin an older Langflow release that depends on a LiteLLM version with a prebuilt Windows wheel: <code>uv pip install --only-</code>

	binary=litellm "langflow==1.10.2", which resolves litellm==1.91.4 and installs cleanly with no compiler needed. Alternative (heavier): install Microsoft C++ Build Tools with the "Desktop development with C++" workload and build 1.11.0. See ADR-002 and Research topic 3.
Date Resolved	2026-07-23

BUG-006 · Live Groq API key committed in plaintext across generated flow exports

Field	Detail
Issue	Several generated flows/*.json exports (the LangGraph RAG Pipeline family) carried a real Groq API key (<redacted>) baked into GroqModel node api_key values, staged for commit.
Found Date	2026-07-27
Status	Closed
Severity	HIGH
File	flows/1.json–flows/9.json, flows/LangGraph RAG Pipeline.json
Description	The Custom-Component/flow generator wrote a live credential straight into node template values instead of a placeholder, so every exported snapshot of the pipeline carried the real key in plaintext JSON.
Root Cause	The generator script's Groq node config used the working key from the local dev session instead of a placeholder string or load_from_db reference.
Solution	Redacted every occurrence to "----" before finalizing the commit (the f8bb1cf "API removed" commit already contains only the redacted value). Recommended follow-up: rotate the exposed key (it did reach git history in f8bb1cf before redaction if git log -p is ever consulted) and switch the field to Langflow's global-variable mechanism (value: "GROQ_API_KEY", load_from_db: true) so future exports never carry a live secret at all.
Date Resolved	2026-07-27

BUG-007 · Fresh import fails Playground pre-flight — "is missing Knowledge" / "is missing Memory Base"

Field	Detail
Issue	Re-importing flows/LangGraph RAG Pipeline.json into a clean Langflow instance failed Langflow's pre-run validation: Knowledge · documents is missing Knowledge, Knowledge · learned_qa is missing Knowledge, Memory Base (feedback + prior turns) is missing Memory Base — despite every node being wired correctly.
Found Date	2026-07-27
Status	Closed
Severity	MEDIUM
File	flows/LangGraph RAG Pipeline.json (Knowledge and Memory Base nodes)
Description	The knowledge_base / memory_base dropdown fields are required and were empty on import, which Langflow's pre-flight "required, visible, no value, no incoming edge" check flags before a run can start.
Root Cause	Knowledge and Memory Base selections are per-user resources living in the Langflow instance's own storage (~/.langflow/knowledge_bases/<user>/<kb_name>/, and a DB row scoped to the current flow ID for Memory Base) — not something a flow JSON export can carry. A fresh import always starts with these fields blank (see Research topic 5).
Solution	Pre-filled the knowledge_base dropdown values (documents, learned_qa) in the generator so only matching Knowledge Bases need to be created and the dropdown auto-resolves; removed the Memory Base node entirely (ADR-008) rather than requiring one to be created and attached to the workflow on every fresh import.
Date Resolved	2026-07-27

BUG-008 · Hugging Face Inference API embeddings — SSRF-blocked DNS resolution

Field	Detail
Issue	Running the flow raised <code>ValueError: SSRF Protection: DNS resolution failed for api-inference.huggingface.co: [Errno 11001] getaddrinfo failed.</code>
Found Date	2026-07-27
Status	Closed
Severity	MEDIUM
File	Environment (Windows DNS configuration), not project code
Description	The Hugging Face Inference API embeddings component could not resolve <code>api-inference.huggingface.co</code> . <code>nslookup api-inference.huggingface.co</code> and <code>nslookup router.huggingface.co</code> both timed out against the configured DNS server (<code>gpon.net</code> , <code>fe80::1</code> — the router).
Root Cause	The Windows network adapter's only DNS server was the router, which was not resolving external hostnames at all; unrelated to Langflow, the flow, or Hugging Face.
Solution	Set the adapter's DNS servers to public resolvers (<code>1.1.1.1 / 8.8.8.8</code>) via Settings → Network & Internet → DNS server assignment, or <code>Set-DnsClientServerAddress -InterfaceAlias "Wi-Fi" -ServerAddresses ("1.1.1.1","8.8.8.8")</code> , then <code>ipconfig /flushdns</code> . In the flow itself, the remote Hugging Face Inference API node was subsequently replaced by a different embeddings provider rather than relying on that endpoint.
Date Resolved	2026-07-27

BUG-009 · Embeddings node with a required, empty API key aborts the run before any Chat Output builds

Field	Detail
Issue	Playground runs showed a fleeting "AI" bubble at the top of the chat history, then reverted to showing only the user's message, with no red-bordered node in the canvas.
Found Date	2026-07-27
Status	Closed
Severity	HIGH
File	flows/LangGraph RAG Pipeline.json (embeddings node feeding generate_query_variants)
Description	Reproduced headlessly via Graph.async_start(): the run aborted with lfx.exceptions.component.ComponentBuildError: Error building Component Google Generative AI Embeddings: API Key is required, raised while building generate_query_variants's embeddings dependency — i.e. on the second vertex of the run, long before any Chat Output could be reached.
Root Cause	Two compounding facts: (1) the standalone embeddings component's api_key field is required and was empty; (2) wiring the optional embeddings handle to any embeddings node makes that node a hard build dependency even though the input itself is declared optional — an unconfigured node on an optional input still blocks the run once an edge exists.
Solution	Either supply a real API key for the connected embeddings provider, or delete the embeddings node and its edges — every consumer (generate_query_variants, dedup_merge_documents, dedup_merge_learned_qa) already falls back to token-overlap similarity when no embeddings model is connected (covered by the test suite in the flow-generation session).
Date Resolved	2026-07-27

BUG-010 · generate_draft never runs — every query silently falls through to no_context_answer

Field	Detail
Issue	With no red error node and a flow that "looked" correct, every Playground query routed to no_context_answer instead of generate_draft, even though the documents Knowledge Base held 106 ingested chunks.
Found Date	2026-07-27
Status	Closed
Severity	HIGH
File	flows/LangGraph RAG Pipeline.json (retrieve and combine_tracks components)
Description	Two independent causes compounded. (a) retrieve's per-track similarity floors (DOCUMENTS_MIN_SIMILARITY = 0.53, LEARNED_QA_MIN_SIMILARITY = 0.57) were compared directly against the Knowledge node's score field, which is a Chroma distance in roughly [-2, 0] (closer to 0 is better), not a [0, 1] similarity — so score >= 0.53 was false for every row and retrieve accepted zero chunks every time (see Research topic 6). (b) Separately, combine_tracks had several conditional outputs (to_draft, to_answer, to_retry, to_no_context) wired alongside other routers into the same downstream input fields; Langflow built an inactive branch's vertex before the active branch had populated it, surfacing as generate_draft has not been built yet (see Research topic 7).
Root Cause	(a) a similarity-vs-distance sign/scale mismatch between the ported Memora thresholds and the Knowledge node's native score convention; (b) multiple conditional-router outputs converging on one input field, which is a build-order hazard in Langflow's dependency resolution.
Solution	(a) convert similarity = 1 + score (Chroma's negated-distance score), clamped to [0, 1], before applying the threshold; if no row in a populated ranked search clears the fixed model-specific floor, retain its two best rows rather than incorrectly treating the Knowledge Base as empty. (b) collapsed the routing to one deterministic path — combine_tracks.to_draft → generate_draft

	→ <code>check_answer_quality.to_answer</code> → <code>generate_answer</code> — removing the retry back-edges into <code>generate_query_variants</code> in the process (ADR-009).
Date Resolved	2026-07-27

BUG-011 · Local MiniLM label did not match the component implementation

Field	Detail
Issue	A later edit intended to switch the embeddings provider to a local, keyless <code>sentence-transformers/all-MiniLM-L6-v2</code> model (confirmed working in a standalone smoke test: 384-dimensional normalized vectors, no network, no key). The node was renamed "Local MiniLM Embeddings" and its <code>model_name</code> field set to <code>sentence-transformers/all-MiniLM-L6-v2</code> . This most plausibly reproduces the persistent "no answer / vanishing AI bubble" symptom reported after that edit.
Found Date	2026-07-27
Status	Closed
Severity	HIGH
File	<code>flows/LangGraph RAG Pipeline.json</code> — node displayed as "Local MiniLM Embeddings"
Description	Verified directly against the exported JSON: the node's <code>type</code> is still "Google Generative AI Embeddings" (module: <code>lfx.components.google.google_generative_ai_embeddings.GoogleGenerativeAIEmbeddingsComponent</code>), with <code>template.api_key.value == ""</code> (required, empty) and <code>template.model_name.value == "sentence-transformers/all-MiniLM-L6-v2"</code> — a Hugging Face model id being sent to Google's embeddings API, which will not recognize it even if a key were supplied.
Root Cause	Editing a node's display label and one field value does not change its underlying component <code>type</code> /module — those are independent in a Langflow export. The edit changed what the node is <i>called</i> and <i>configured with</i> , not what code actually runs, so it still hits the same class of failure as BUG-009 (required

	empty <code>api_key</code>), now compounded by an invalid <code>model_name</code> for the provider it actually is.
Solution	Replaced the runtime code with <code>langchain_huggingface.HuggingFaceEmbeddings</code> , configured for <code>sentence-transformers/all-MiniLM-L6-v2</code> , <code>local_files_only=True</code> , and normalized output. Removed the unused remote Hugging Face inference node. A smoke test produced normalized 384-dimensional vectors, and <code>Graph.from_payload</code> validated the updated component.
Date Resolved	2026-07-27